How commands reach the motors

Two private applications, two buses and two bridges; only serialized frames cross between cores.

M7 · UI controller · 480 MHz

CLI + GUI
Commands and presentation

→

Motion client
Tracks requests and job history

↔

Planning
Conversions, paths and profiles

↕

Private pneumo bus ↔ ICC Bridge

One ICC coroutine context / worker (priority 4). All three lanes dispatch here.

M4 · IO controller · 240 MHz

Controller
Admission and ownership

→

Executor
Profiles → pulse schedules

→

TIM2 + DMA
Three physical STEP outputs

↕

Private pneumo bus ↔ ICC Bridge

One ICC coroutine context / worker (priority 4). Execution runs in separate workers.

↕ encode / decode

↕ encode / decode

Shared SRAM4 · three independent queues in each direction

6 queues total

URGENT
Stop, cancel, lease and link control
Up to 2,048 bytes per frame
▢▢▢▢ 4 slots per direction

NORMAL
Preparation, actions, replies and state
Up to 2,048 bytes per frame
▢▢▢▢ 4 slots per direction

BACKGROUND
IO logs → UI → USB console
Up to 512 bytes per frame
▢▢▢▢ 4 slots per direction

HSEM is the doorbell: it wakes a notification worker. The message bytes remain in shared memory.

What crosses the bridge

Services	request → deferred reply
Actions	acceptance → progress → terminal result
Topics	latest motor state at 20 Hz; bounded logs

Routes are declared in common/icc/catalog/{services,actions,topics}.hpp.

Separate budgets, shared dispatch

Urgent traffic has its own capacity, but it still needs CPU time in the same ICC context.

UI lease: renew every 50 ms · expires after 250 ms

M4 execution: 40 ms renewal · 80 ms commit frontier

Sources: ICC catalogs / Bridge / InterCoreTransport; hal/util/InterCoreMemory.hpp; motion/OperatingEnvelope.hpp

Memory: storage, execution and working space

Logical allocations, not to scale. RAM usage below belongs to the normal Release linked image.

M4 private RAM

D2 SRAM · 288 KiB total

Code alias 0x10000000 = physical 0x30000000

2 KiB vectors · 0x10000000

192 KiB fast code + constants

0x10000800 · 193,248 B linked

86 KiB data + DMA buffers

0x30030800 · 82,016 B linked

8 KiB interrupt stack · 0x30046000

AXI heap region · 256 KiB

0x24040000 → 0x24080000

Unwind metadata precedes the heap.
Heap capacity is checked at runtime.

The SRAM alias is not additional memory.

M7 private RAM

ITCM · 64 KiB

16,088 B code + 24,000 B GUI data

DTCM · 128 KiB

0x20000000: vectors, 2 KiB
0x20000800: data, 110 KiB
0x2001C000: gap, 8 KiB
0x2001E000: interrupt stack, 8 KiB

AXI private region · 256 KiB

0x24000000 → 0x24040000

Static data and ordinary heap.
Heap capacity is checked at runtime.

Thread stacks use RAM too

The UI planner has a static DTCM stack. Other UI workers consume private RAM.

An 8 KiB interrupt stack is not a thread pool.

Shared / external RAM

SRAM4 · 64 KiB

0x38000000 · ICC reserve: 60 KiB
49,312 B used; final 4 KiB: boot handoff

External SDRAM · 8 MiB

Framebuffers · 4 MiB reserved

0xC0000000 · non-cacheable
Two images, 1,228,800 B each

M4 code + profiling · 2 MiB

0xC0480000 · code: 1536 KiB
0xC0400000 · profiling: 512 KiB

UI working arena · 2 MiB

0xC0600000 · cacheable
3D colour/depth buffers: 1,500 KiB
Planning arena: 256 KiB

Spare SDRAM does not automatically enlarge either private heap.

Internal flash · 2 MiB

M7 bootstrap: 0x08000000 · 128 KiB
M4 bootstrap: 0x08100000 · 128 KiB
File storage: 0x08180000 · 512 KiB

Bootstrap initializes memory and copies RAM sections before starting both applications.

External QSPI flash · 16 MiB · images survive power-off

1	M4 · 3 MiB	M7 image + GUI assets · 10 MiB	1	1

0x90000000 · Factory area · 1 MiB
0x90100000 · M4 image · 3 MiB
0x90400000 · M7 image · 10 MiB

Code, meshes, fonts and images together
0x90E00000 · Manifest / scratch · 1 MiB
0x90F00000 · Factory area · 1 MiB

Current source • Normal Release pair: 6bb2cd35b1b7e73a • Electrical tests do not establish mechanical qualification

Sources: cmake/application_giga.cmake; boot linker/manifest; normal Release manifest. Heap capacity excludes linker/runtime metadata.

Which functions run from faster memory?

Normal Release execution addresses; QSPI also holds the load copies of code that runs in RAM.

M4 · execution and communication

Internal SRAM · fast code region

`0x10000800 · 192 KiB reserved`

STEP generation
StepGenerator, StepHardware, StepOutput
Motion evaluation
Executor, ProfileSchedule, Profile, Sequence
Motor / safety paths
StepperMotor, feedback, driver fault handling
Communication paths
Bridge, ServiceBridge, ActionBridge, transport
Runtime / helpers
ThreadX, clock, notification, selected libc/libgcc

193,248 / 196,608 B · 98.29% · 3,360 B remain

SDRAM · ordinary M4 code

`0xC0480000 · 1536 KiB reserved`

Server dispatch, admission, worker construction and cold helpers now run from SDRAM. Bootstrap copies them from QSPI. STEP and interrupt call paths remain in internal SRAM.

QSPI · remaining read-only material

Ordinary constants generally remain in flash; constants selected with fast code move to internal SRAM.

M7 · presentation and planning

ITCM · selected TGX renderer routines

`0x00000000 · 16,088 B linked`

`_drawMesh uber_shader _discardBox`

Selected geometry / raster paths avoid external instruction fetches. The current allocation is below the 24 KiB renderer cap.

QSPI + M7 instruction cache

`Ordinary application starts at 0x90400000`

GUI, CLI, Robot and Planner code execute here. The M7 caches instructions; the M4 does not have an equivalent instruction cache.

DTCM · planning stack and CPU data

The planning worker has a static 24 KiB DTCM stack. Its large planning arena is in SDRAM.

A fast stack does not place the planner’s functions in ITCM.

Important configuration distinction

Normal Debug, Release and profiling builds enable M4 SDRAM code, as does the electrical fixture. The first 512 KiB of this SDRAM allocation remains profiling data, execute-never. Linked audits check internal-RAM interrupt paths; they are not timing guarantees.

Sources: cmake/giga_fast_code.cmake; application_giga.cmake; linked ELF symbols / manifest; CMake cache for flashed test pair.

Motion baseline: evidence and remaining limits

Software correctness, electrical timing and mechanical qualification are separate questions.

Admission and ownership

One exclusive Motor / Axis / Robot family. Reservations include preparation and cleanup. Protocol, configuration, session, axis revisions and positions are checked before outputs are armed.

Execution and stopping

M4 maintains immutable schedules with an installed brake. Explicit stop and lease loss select that branch locally. Abort / guard inhibition are separate fault paths and can invalidate referencing.

Memory remains a constraint

Normal M4 fast code has 3,360 B free; the profiling build has only 56 B. Raw/homing workers reuse 10 KiB stacks. Commands have 28 KiB and the executor 8 KiB. The short IO workflow retained at least 29% measured stack headroom.

Concurrent motion: ICC allocation fixed

Senders reuse their frame buffers. M4 admission/dispatch now run from SDRAM after traces identified QSPI stalls. Independent-axis overlaps and the complete buffered-raw workflow pass without deadline or priority changes.

Startup and stopping: verified

Initial DMA calculations run with interrupts available in bounded batches; publication rechecks cancellation/revision. Final captures pass raw 800/800/400, coordinated 400/400/400 and early stop 266/266/266, with enable inactive.

Verification scope

Clang-tidy: 430/430 contexts, zero findings. Internal-RAM call-path audit passes. Full host and sanitizer outcomes are recorded in the current review. Short electrical tests do not qualify maximum-rate endurance, powered mechanics or the physical panel.

Use the review report for exact test outcomes and capture hashes. No diagram claims that a passing short capture proves a full-rate or mechanically safe operating envelope.

Sources: Controller / Ownership / StepperMotor; OperatingEnvelope; review report and hardware captures.